Ambiente de IA Local — LM Studio + Qwen

Versão: 0.1
Máquina: Intel i5-12600KF / RTX 5060 8 GB / 16 GB RAM
Runtime: LM Studio / llmster
Objetivo: utilizar modelos locais diferentes conforme carga da máquina e complexidade da tarefa.

---

1. Estratégia geral

Não teremos um único modelo para tudo.

A estratégia será manter modelos com funções diferentes:

Perfil	Modelo	Função
LIGHT	Qwen3 4B Q4_K_M	tarefas simples e rápidas
WORK	**Qwen3 8B Q4_K_M**	modelo principal
HEAVY	Qwen3 14B Q4_K_M	tarefas complexas com PC livre
CODER HEAVY	Qwen3-Coder-30B-A3B	futuro, após upgrade
EXPERIMENTAL	Qwen3-Coder-Next	futuro, hardware maior

Não é necessário manter todos carregados.

Os modelos podem permanecer **baixados no disco**, mas apenas o modelo atualmente carregado ocupa RAM/VRAM.

---

2. Qwen3 4B — LIGHT

Objetivo

Modelo pequeno para tarefas auxiliares.

GGUF Q4_K_M:

```
~2,5 GB
```

O Qwen disponibiliza oficialmente essa quantização.

Bom para

Classificação de textos, geração de pequenas respostas, documentação simples, resumo de informações, transformação de JSON, geração de pequenas funções, pequenos testes unitários, agentes auxiliares e tarefas que não exijam raciocínio profundo.

Programação

Exemplo adequado:

```
Crie um DTO Java para esta entidade.

Crie testes para esta classe.

Transforme este JSON em um record Java.

Explique este método.

Crie uma migration SQL simples.
```

Aplicação inteira?

Não é o modelo indicado.

Ele provavelmente consegue gerar a estrutura de uma aplicação pequena, mas começará a cometer inconsistências quando precisar manter arquitetura, regras de negócio, vários arquivos e decisões anteriores.

Quando usaremos

Quando você estiver trabalhando e quiser consumir o mínimo possível da máquina.

Perfil:

```
qwen-light
```

---

3. Qwen3 8B — WORK

Nosso modelo principal

Este é o modelo que já instalamos.

Configuração atual:

```
Qwen3 8B
Q4_K_M
Contexto: 8192
GPU Offload: 100%
```

Nosso teste estimou:

```
GPU: 4.68 GiB
RAM adicional estimada: praticamente zero
```

Isso é excelente para a RTX 5060 8 GB.

Bom para

Desenvolvimento Java/Spring Boot, Angular, SQL, APIs REST, análise de bugs, geração de testes, documentação técnica, refatorações moderadas, criação de endpoints, pequenas features completas, análise de código e execução de Skills.

Também será nosso primeiro modelo para agentes.

Exemplos

```
Implemente uma feature de cadastro de cartão.

Analise este erro do Hibernate.

Crie Controller + Service + Repository.

Revise este Pull Request.

Procure possíveis bugs nesta classe.

Implemente esta GitHub Issue.
```

Ele consegue criar uma aplicação inteira sozinho?

Tecnicamente consegue gerar uma aplicação pequena. Não significa que seja confiável deixá-lo fazer tudo em um único prompt.

Por exemplo:

```
Crie uma API Spring Boot
com autenticação,
PostgreSQL,
```

```
CRUD de usuários
e Docker.
```

Ele provavelmente consegue produzir algo utilizável.

Mas esta não será nossa estratégia.

Usaremos:

```
Issue
 ↓
Planejamento
 ↓
Implementação
 ↓
Compilação
 ↓
Testes
 ↓
Code Review
 ↓
Correção
```

Assim o 8B trabalha em pequenos ciclos.

Nesse formato ele pode, sim, contribuir para construir **uma aplicação inteira**, feature por feature.

A inteligência do sistema não estará apenas no modelo.

Será:

```
Modelo
+
Skill
+
Harness
+
Ferramentas
+
Testes
+
Git
+
Feedback
```

Essa combinação é muito mais importante para autonomia do que simplesmente aumentar o número de parâmetros.

Quando usaremos

Este será o padrão para aproximadamente a maior parte das tarefas locais.

Perfil:

```
qwen-work
```

---

4. Qwen3 14B — HEAVY

Q4_K_M:

```
~9 GB
```

Portanto ele já ultrapassa os 8 GB físicos da nossa RTX.

Parte do modelo precisará permanecer na RAM.

Bom para

Arquitetura, análise de código mais complexa, refatorações grandes, raciocínio sobre várias classes, investigação de bugs difíceis, planejamento de features e revisão de implementações produzidas pelo 8B.

Uma estratégia interessante será:

```
Qwen 14B
   ↓
planeja

Qwen 8B
   ↓
implementa

Qwen 14B
   ↓
revisa
```

Ou inverter funções após benchmarks.

Aplicação inteira?

Ele deverá ser mais consistente que o 8B em tarefas complexas, mas ainda não deve receber algo do tipo:

```
Faça meu sistema inteiro e me avise quando terminar.
```

O correto continua sendo trabalhar em ciclos.

Quando usaremos

Principalmente quando:

```
PC ocioso
IDE fechada ou pouco utilizada
sem jogos
sem aplicações pesadas
```

Antes de adotá-lo definitivamente, faremos:

```
lms load --estimate-only qwen/qwen3-14b
    --gpu max
    --context-length 8192
```

E analisaremos o consumo real.

Perfil planejado:

```
qwen-heavy
```

---

5. Qwen3-Coder-30B-A3B — CODER HEAVY

Este é particularmente interessante para o projeto futuro de agentes.

É um modelo Mixture-of-Experts voltado para código.

A versão Q4_K_M ocupa aproximadamente:

```
18,6 GB
```

O problema atual é memória.

Temos:

```
VRAM: 8 GB
RAM: 16 GB
```

e o Ubuntu já utiliza parte da RAM.

Portanto:

```
NÃO INSTALAR AGORA
```

Bom para

Agentes autônomos de programação, implementação de Issues, uso de ferramentas, refatorações grandes, análise de repositórios e tarefas de desenvolvimento mais longas.

Quando testar

Após aproximadamente:

```
32 GB RAM
+
mais espaço de SSD
```

Esse provavelmente será um dos primeiros modelos pesados que avaliaremos após upgrade.

---

6. Qwen3-Coder-Next — FUTURO

Modelo voltado especificamente para coding agents e desenvolvimento local.

Ele possui:

```
80B parâmetros totais
~3B parâmetros ativos
256K contexto nativo
```

O fato de ativar somente aproximadamente 3B parâmetros por token ajuda na inferência, mas **todos os pesos ainda precisam existir em memória/armazenamento**.

Portanto não é adequado ao hardware atual.

Ele ficará apenas na nossa lista de modelos futuros.

---

7. Resumo de decisão

Modelo	Qualidade	Velocidade	Hardware atual	Uso
Qwen3 4B	★★★	★★★★★	Excelente	tarefas simples
Qwen3 8B	★★★★	★★★★	**Excelente**	modelo principal
Qwen3 14B	★★★★½	★★★	Aceitável	tarefas pesadas
Coder 30B-A3B	★★★★★	★★	Ruim atualmente	futuro agente coder
Coder-Next	★★★★★	—	Inadequado	futuro

---

8. Ver modelos instalados

```
lms ls
```

Isso mostra modelos armazenados no disco.

Modelo armazenado no disco **não significa modelo carregado**.

---

9. Ver modelos carregados

```
lms ps
```

Esse comando mostra os modelos atualmente residentes em memória.

Se aparecer:

```
No models are currently loaded.
```

nenhum modelo está consumindo VRAM/RAM para inferência.

---

10. Carregar nosso Qwen principal

Configuração atual:

```
lms load qwen/qwen3-8b
    --gpu max
```

```
    --context-length 8192
    --identifier qwen-work
```

O `--identifier` fornece um nome estável para utilizarmos depois pela API. O LM Studio suporta identificação, contexto, GPU offload e TTL durante o carregamento.

Depois:

```
lms ps
```

---

11. Conversar diretamente com ele

```
lms chat qwen-work
```

---

12. Descarregar um modelo

Pode-se executar:

```
lms unload
```

e selecionar o modelo.

Ou descarregar todos:

```
lms unload --all
```

Este será o comando que utilizaremos frequentemente.

---

13. Alternar modelos

A forma recomendada no nosso PC é:

```
lms unload --all
```

e depois carregar o próximo.

Por exemplo:

```
lms load qwen/qwen3-8b
    --gpu max
    --context-length 8192
    --identifier qwen-work
```

No futuro:

```
lms unload --all

lms load qwen/qwen3-14b
    --gpu max
    --context-length 8192
    --identifier qwen-heavy
```

Não precisamos reiniciar o LM Studio.

---

14. Posso ter vários modelos carregados?

Sim.

O LM Studio suporta múltiplas instâncias e múltiplos modelos carregados simultaneamente.

Por exemplo:

```
Qwen 4B
+
Qwen 8B
```

poderiam permanecer carregados ao mesmo tempo.

No nosso PC isso geralmente será uma má ideia.

Com apenas:

```
8 GB VRAM
16 GB RAM
```

os modelos começariam a disputar memória.

Nossa regra será:

```
1 modelo carregado por vez
```

salvo experimentos específicos.

---

15. Vários agentes não precisam de vários modelos

Muito importante:



Todos podem utilizar o mesmo modelo.

O LM Studio suporta requisições concorrentes por continuous batching.

Portanto não precisamos fazer:

```
4 agentes
=
4 Qwens carregados
```

Podemos fazer:

```
4 agentes
=
1 Qwen compartilhado
```

Isso será essencial para nosso futuro harness.

---

16. Auto-unload

Podemos carregar um modelo com TTL:

```
lms load qwen/qwen3-8b
    --gpu max
    --context-length 8192
    --identifier qwen-work
    --ttl 600
```

Isso significa:

```
10 minutos sem requisição
        ↓
modelo descarregado automaticamente
```

O TTL é suportado nativamente pelo LM Studio.

Isso será útil para Skills e agentes ocasionais.

Para nosso ambiente automatizado, provavelmente utilizaremos TTL.

---

17. Servidor da API

Para disponibilizar o Qwen para aplicações:

```
lms server start
```

Parar:

```
lms server stop
```

Servidor padrão:

```
localhost:1234
```

Ter o servidor ligado não significa necessariamente que existe um modelo carregado.

---

18. Daemon llmster

Iniciar:

```
lms daemon up
```

Status:

```
lms daemon status
```

Parar:

```
lms daemon down
```

O daemon será importante quando começarmos a automatizar agentes.

---

19. Modo "quero jogar"

Quando você quiser que a IA local libere os recursos da máquina:

```
lms unload --all
```

Isso é suficiente para liberar os modelos.

Mas como nossa exigência é **não deixar nenhuma infraestrutura de IA local rodando em background**, utilizaremos:

```
lms unload --all

lms server stop

lms daemon down
```

Depois:

```
nvidia-smi
```

A VRAM utilizada pelo Qwen deverá desaparecer.

Você verá apenas coisas como:

```
Xorg
GNOME
Chrome
Steam
jogo
```

e não o modelo.

---

20. Atenção caso a interface gráfica esteja aberta

`lms daemon down` encerra o **llmster**.

Ele não encerra automaticamente o aplicativo gráfico do LM Studio.

Portanto, se o LM Studio GUI estiver aberto:

```
Fechar LM Studio GUI
+
lms unload --all
+
lms server stop
+
lms daemon down
```

é nosso estado:

```
LOCAL AI = OFF
```

---

21. Modo operacional futuro

Criaremos posteriormente comandos nossos:

```
ia light
ia work
ia heavy
ia off
ia status
```

Comportamento pretendido:

```
ia light
    ↓
Qwen3 4B

ia work
    ↓
Qwen3 8B

ia heavy
```

```
    ↓
Qwen3 14B

ia off
    ↓
unload all
server stop
daemon down

ia status
    ↓
modelo
VRAM
RAM
servidor
daemon
```

Assim você não precisará decorar os comandos do LM Studio.

---

22. Regra para baixar novos modelos

Nunca baixar simplesmente porque o modelo parece interessante.

Primeiro:

```
lms load --estimate-only <modelo>
    --gpu max
    --context-length 8192
```

Depois analisar:

```
VRAM
RAM
espaço em disco
velocidade
qualidade
```

Só então adicionar o modelo ao nosso ambiente.

---

23. Estado atual

Neste momento:

```
LM Studio / llmster
        ✅ instalado

Qwen3 8B Q4_K_M
        ✅ baixado

Estimativa
        ✅ concluída

GPU Offload
        100%

VRAM estimada
        4.68 GiB

Contexto inicial
        8192

Modelo carregado
        ❌ ainda não
```

Nosso próximo passo será carregar o `qwen-work`, medir desempenho real e criar os primeiros comandos de gerenciamento.